

## HACK THE BOX SHERLOCK - DFIR CROWNJEWEL-1

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ ls
C Microsoft-Windows-NTFS.evtx SECURITY.evtx SYSTEM.evtx

(rrafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ cd C

(rrafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts/C]
$ ls
'$MFT'
```

Upon unzipping the file downloaded from HacktheBox, there are a lot of evtx files which are Windows Event Logs and an \$MFT file

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ ls
C Microsoft-Windows-NTFS.evtx ntfs.json SECURITY.evtx security.json SYSTEM.evtx system.json
```

I converted the evtx files into JSON files each using Chainsaw dump.

1. Attackers can abuse the vssadmin utility to create volume shadow snapshots and then extract sensitive files like NTDS.dit to bypass security mechanisms. Identify the time when the Volume Shadow Copy service entered a running state.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ strings security.json | grep -i "volume shadow copy"

(rrafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ strings system.json | grep -i "volume shadow copy"
"param1": "Volume Shadow Copy",

(rrafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ strings ntfs.json | grep -i "volume shadow copy"
```

I searched for the string "Volume Shadow Copy" in all 3 JSON files using strings and grep CLI utility.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ jq '.[] | select(.Event.EventData.param1 == "Volume Shadow Copy")' system.json
{
  "Event_attributes": {
    "xmlns": "http://schemas.microsoft.com/win/2004/08/events/event"
  },
  "Event": {
    "System": {
      "Provider_attributes": {
        "Name": "Service Control Manager",
        "Guid": "{555908d1-a6d7-4695-8e1e-26931d2012f4}",
        "EventSourceName": "Service Control Manager"
      },
      "EventID_attributes": {
        "Qualifiers": 16384
      },
      "EventID": 7036,
      "Version": 0,
      "Level": 4,
      "Task": 0,
      "Opcode": 0,
      "Keywords": "0x8080000000000000",
      "TimeCreated_attributes": {
        "SystemTime": "2024-05-14T03:42:16.783105Z"
      },
      "EventRecordID": 2984,
      "Correlation": null,
      "Execution_attributes": {
        "ProcessID": 784,
        "ThreadID": 916
      },
      "Channel": "System",
      "Computer": "DC01.forela.local",
      "Security": null
    },
    "EventData": {
      "param1": "Volume Shadow Copy",
      "param2": "running",
      "Binary": "5600530053002F0034000000"
    }
  }
}
```

Then I used jq to filter by the attributes down to the param1 like shown in the previous screenshot. I filtered it to the string of "Volume Shadow Copy" and there I can see one event, and the system time is **2024-05-14 03:42:16**

2. When a volume shadow snapshot is created, the Volume shadow copy service validates the privileges using the Machine account and enumerates User groups. Find the two user groups the volume shadow copy process queries and the machine account that did it.

Google search results for the query: "evt log and make filter 4799 this id related to enumerates local user groups to validate privileges, often using a machine account, is 4799". The search results show a primary Event ID of 4799 (Security Group Enumeration) and associated events like 7036 (System log, shows Volume Shadow Copy entering running state). The context is used to identify when the machine account (e.g., WORKSTATION\$) queries groups like "Backup Operators" or "Administrators" to dump ntfs.dit.

AI Overview **Indonesia**

The primary Event ID for detecting when a process (such as Volume Shadow Copy) enumerates local user groups to validate privileges, often using a machine account, is **4799**. This event is recorded in the Security log and indicates "A security-enabled local group membership was enumerated". [Medium +2](#)

- **Key Event ID: 4799** (Security Group Enumeration)
- **Associated Events: 7036** (System log, shows Volume Shadow Copy entering running state)
- **Context:** Used to identify when the machine account (e.g., WORKSTATION\$) queries groups like "Backup Operators" or "Administrators" to dump ntfs.dit. [Medium +2](#)

[Show more](#)

**CrownJewel-1 Sherlock HackTheBox Medium**  
5 Jan 2025 — evt log and make filter 4799 this id related to enumerates local user groups to validate privileges, often using a machine account, is 4799. [Medium](#) · [Mahmouddesokey](#)

**CrownJewel-1 - Whoami | FaresMorad**  
20 Apr 2025 — Get-WinEvent -Path &SYSTEM.evtx | Where-Object {\$\_.Name -eq "4799"} [Medium](#)

I Googled what EventID would be appropriate for this question and found out to be 4799 and I filtered this down to the attributes using jq. And then I found the results as these below.

```
"EventData": {
  "TargetUserName": "Backup Operators",
  "TargetDomainName": "Builtin",
  "TargetSid": "S-1-5-32-551",
  "SubjectUserSid": "S-1-5-18",
  "SubjectUserName": "DC01$",
  "SubjectDomainName": "FORELA",
  "SubjectLogonId": "0x3e7",
  "CallerProcessId": "0x1398",
  "CallerProcessName": "C:\\Windows\\System32\\svchost.exe"
}
```

```
"EventData": {
  "TargetUserName": "Administrators",
  "TargetDomainName": "Builtin",
  "TargetSid": "S-1-5-32-544",
  "SubjectUserSid": "S-1-5-18",
  "SubjectUserName": "DC01$",
  "SubjectDomainName": "FORELA",
  "SubjectLogonId": "0x3e7",
  "CallerProcessId": "0x1398",
  "CallerProcessName": "C:\\Windows\\System32\\svchost.exe"
}
```

```
},
"EventData": {
  "TargetUserName": "Administrators",
  "TargetDomainName": "Builtin",
  "TargetSid": "S-1-5-32-544",
  "SubjectUserSid": "S-1-5-18",
  "SubjectUserName": "DC01$",
  "SubjectDomainName": "FORELA",
  "SubjectLogonId": "0x3e7",
  "CallerProcessId": "0x1398",
  "CallerProcessName": "C:\\Windows\\System32\\svchost.exe"
}
}
```

I found that the group names are Backup Operators and Administrators, while the machine account name is DC01\$. So following the format HacktheBox gave, I followed and it is **Administrators, Backup Operators, DC01\$**

3. Identify the Process ID (in Decimal) of the volume shadow copy service process.

[illegible]

Then I used the jq command but I used the syntax to get all the full events' details that have that keyword.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ jq '.[0] | select(tostring | contains("VSS"))' security.json
{
  "Event_attributes": {
    "xmlns": "http://schemas.microsoft.com/win/2004/08/events/event"
  },
  "Event": {
    "System": {
      "Provider_attributes": {
        "Name": "Microsoft-Windows-Security-Auditing",
        "Guid": "54849625-5478-4994-A5BA-3E3B0328C30D"
      },
      "EventID": 4799,
      "Version": 0,
      "Level": 0,
      "Task": 13826,
      "Opcode": 0,
      "Keywords": "0x8020000000000000",
      "TimeCreated_attributes": {
        "SystemTime": "2024-05-14T03:42:16.793460Z"
      },
      "EventRecordID": 5900,
      "Correlation": null,
      "Execution_attributes": {
        "ProcessID": 828,
        "ThreadID": 1120
      },
      "Channel": "Security",
      "Computer": "DC01.forela.local",
      "Security": null
    },
    "EventData": {
      "TargetUserName": "Administrators",
      "TargetDomainName": "Builtin",
      "TargetSid": "S-1-5-32-544",
      "SubjectUserSid": "S-1-5-18",
      "SubjectUserName": "DC01$",
      "SubjectDomainName": "FORELA",
      "SubjectLogonId": "0x3e7",
      "CallerProcessId": "0x1190",
      "CallerProcessName": "C:\\Windows\\System32\\VSSVC.exe"
    }
  }
}
```

Then we saw it had the CallerProcessId of 0x1190. Then I will ask ChatGPT to change it to decimal since the question asked for decimal.

Convert hexadecimal 0x1190 to decimal:

$$1 \times 16^3 + 1 \times 16^2 + 9 \times 16^1 + 0 \times 16^0 = 4096 + 256 + 144 + 0 = 4496$$

Answer: 4496

And we get the answer **4496**

4. Find the assigned Volume ID/GUID value to the Shadow copy snapshot when it was mounted.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ strings system.json | grep -i "shadow copy"
  "param1": "Volume Shadow Copy",
```

I tried to filter for the words like "snapshot" and I eventually found it when I filtered the word shadow copy on system.json (System.evtx originally). And I used the jq again like the previous question.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts]
$ jq '.[ ] | select(tostring | contains("Volume Shadow Copy"))' system.json
{
  "Event_attributes": {
    "xmlns": "http://schemas.microsoft.com/win/2004/08/events/event"
  },
  "Event": {
    "System": {
      "Provider_attributes": {
        "Name": "Service Control Manager",
        "Guid": "{555908d1-a6d7-4695-8e1e-26931d2012f4}",
        "EventSourceName": "Service Control Manager"
      },
      "EventID_attributes": {
```

And there I get the GUID and I thought it was the answer which is, {555908d1-a6d7-4695-8e1e-26931d2012f4}, but it was wrong so I continued my search.



```
},
"EventData": {
  "VolumeCorrelationId": "06C4A997-CCA8-11ED-A90F-000C295644F9",
  "VolumeIdLength": 0,
  "VolumeId": "",
  "VolumeLabelLength": 0,
  "VolumeLabel": "",
  "DeviceNameLength": 33,
  "DeviceName": "\\Device\\HarddiskVolumeShadowCopy1",
  "DeviceGuid": "00000000-0000-0000-0000-000000000000",
  "VendorIdLength": 0,
  "VendorId": "",
  "DeviceNameLength": 33,
  "DeviceName": "\\Device\\HarddiskVolumeShadowCopy1",
  "DeviceGuid": "00000000-0000-0000-0000-000000000000",
  "VendorIdLength": 0,
  "VendorId": ""
}
```

Until I filtered on the Events who have VolumeCorrelationId on ntfs.json and get that one since it has a device name of HarddiskVolumeShadowCopy1 as described in the question so the answer is **{06c4a997-cca8-11ed-a90f-000c295644f9}**

5. Identify the full path of the dumped NTDS database on disk.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts/C]
$ chainsaw dump '$MFT' --json > mft.json

CHAINSAW

By WithSecure Countercept (@FranticTyping, @AlexKornitzer)

[+] Dumping the contents of forensic artefacts from: $MFT (extensions: *)
[+] Loaded 1 forensic artefacts (100.5 MiB)
[+] Done
```

I used the Chainsaw dump CLI tool again on the \$MFT file.

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts/C]
$ strings mft.json | grep -i "NTDS"
"FullPath": "Windows/NTDS",
"FullPath": "Windows/NTDS/ntds.dit",
"FullPath": "Windows/NTDS/edb00001.log",
"FullPath": "Windows/NTDS/edb.log",
"FullPath": "Windows/NTDS/edbres00001.jrs",
"FullPath": "Windows/NTDS/edbres00002.jrs",
"FullPath": "Windows/NTDS/edb.chk",
"FullPath": "Windows/NTDS/ntds.jfm",
"FullPath": "Windows/NTDS/edbtmp.log",
```

Then I saw there are so many

```
FullPath": "Windows/WinSxS/amd64_microsoft-windows-d...oryservices-ntdsatq_31bf3856ad364e35_10.0.20348.1_none_58290689ba5621cc/ntdsatq.dll",  
FullPath": "Users/Administrator/Documents/backup_sync_dc/ntds.dit",  
FullPath": "Windows/WinSxS/X861F5-1.469/ntdsutil.exe",  
FullPath": "Windows/WinSxS/x86_microsoft-windows-d...ommandline-ntdsutil_31bf3856ad364e35_10.0.20348.469_none_67b7f92801516309"
```

Then I saw that one, ntds.dit and I speculated that was the file, and adding the C drive and changing the signs, it becomes

**C:\Users\Administrator\Documents\backup\_sync\_Dc\Ntds.dit.**

6. When was newly dumped ntds.dit created on disk?

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts/C]  
$ jq '.[] | select(tostring | contains("Users/Administrator/Documents/backup_sync_dc/ntds.dit"))' mft.json  
{  
  "Signature": "FILE",  
  "EntryId": 97945,  
  "Sequence": 6,  
  "BaseEntryId": 0,  
  "BaseEntrySequence": 0,  
  "HardLinkCount": 1,  
  "Flags": "EntryFlags(ALLOCATED)",  
  "UsedEntrySize": 344,  
  "TotalEntrySize": 1024,  
  "FileSize": 16777216,  
  "IsADirectory": false,  
  "IsDeleted": false,  
  "HasAlternateDataStreams": false,  
  "StandardInfoFlags": "FileAttributeFlags(FILE_ATTRIBUTE_ARCHIVE)",  
  "StandardInfoLastModified": "2023-03-27T14:02:43.932074Z",  
  "StandardInfoLastAccess": "2024-05-14T03:44:22.844960Z",  
  "StandardInfoCreated": "2024-05-14T03:44:22.813756Z",  
  "FileNameFlags": "FileAttributeFlags(FILE_ATTRIBUTE_ARCHIVE)",  
  "FileNameLastModified": "2024-05-14T03:44:22.813756Z",  
  "FileNameLastAccess": "2024-05-14T03:44:22.813756Z",  
  "FileNameCreated": "2024-05-14T03:44:22.813756Z",  
  "FullPath": "Users/Administrator/Documents/backup_sync_dc/ntds.dit",  
  "DataStreams": []  
}
```

Filtering with the answer from the previous question and using jq, I was able to find the FileNameCreated which means when the file was created for the ntds.dit file and I got the date and time so the answer is

**2024-05-14 03:44:22.**

7. A registry hive was also dumped alongside the NTDS database. Which registry hive was dumped and what is its file size in bytes?

```
(rafael@raf)-[~/HTB_SHERLOCKS/CrownJewel1/Artifacts/C]  
$ strings mft.json | grep -i "backup"  
FullPath": "Users/Administrator/Documents/backup_sync_dc",  
FullPath": "Program Files/WindowsApps/MutableBackup",  
FullPath": "Windows/System32/LogFiles/WMI/RtBackup/EtwRTDefenderAuditLogger.etl",  
FullPath": "Windows/System32/LogFiles/WMI/RtBackup/EtwRTDefenderApiLogger.etl",  
FullPath": "ProgramData/Microsoft/Windows Defender/Definition Updates/Backup",  
FullPath": "ProgramData/Microsoft/Windows Defender/Definition Updates/NisBackup",  
FullPath": "Users/Administrator/Documents/backup_sync_dc/SYSTEM",  
FullPath": "Windows/System32/dns/backup/dns.log",
```

I filtered using strings and grep for backup and found the exact same directory as question number 5, and we see its name is SYSTEM, the registry hive that was dumped.



Rafael Angelo Christianto

And after getting its size, I can use jq again.

```
(rafael@raf) - [~/HTB_SHERLOCKS/CrownJewel1/Artifacts/C]
$ jq '.[0] | select(tostring | contains("Users/Administrator/Documents/backup_sync_dc/SYSTEM"))' mft.json

{
  "Signature": "FILE",
  "EntryId": 663,
  "Sequence": 4,
  "BaseEntryId": 0,
  "BaseEntrySequence": 0,
  "HardLinkCount": 1,
  "Flags": "EntryFlags(ALLOCATED)",
  "UsedEntrySize": 336,
  "TotalEntrySize": 1024,
  "FileSize": 17563648,
  "IsADirectory": false,
  "IsDeleted": false,
  "HasAlternateDataStreams": false,
  "StandardInfoFlags": "FileAttributeFlags(FILE_ATTRIBUTE_ARCHIVE)",
  "StandardInfoLastModified": "2023-03-25T14:40:36.021159Z",
  "StandardInfoLastAccess": "2024-05-14T03:44:42.485735Z",
  "StandardInfoCreated": "2024-05-14T03:44:42.344933Z",
  "FileNameFlags": "FileAttributeFlags(FILE_ATTRIBUTE_ARCHIVE)",
  "FileNameLastModified": "2024-05-14T03:44:42.344933Z",
  "FileNameLastAccess": "2024-05-14T03:44:42.344933Z",
  "FileNameCreated": "2024-05-14T03:44:42.344933Z",
  "FullPath": "Users/Administrator/Documents/backup_sync_dc/SYSTEM",
  "DataStreams": []
}
```

Then I get the file size and the answer is

**SYSTEM, 17563648**



## You have solved CrownJewel-1!

Congratulations  **rafaelangelo** best of luck in capturing flags ahead!

|               |                    |                |
|---------------|--------------------|----------------|
| <b>#2946</b>  | <b>21 Apr 2026</b> | <b>Retired</b> |
| Sherlock Rank | Solve Date         | Sherlock State |

Ok

Share